

《企业级应用软件设计与开发》期末大作业报告

CodeSentinel — 智能代码审查与 Bug 修复 Agent

基于 LangGraph 状态机与对抗式验证的 Agentic AI 工程实践

字段	内容
课程名称	企业级应用软件设计与开发（AI 驱动的软件开发与 Agentic AI）
课程代码	50120224001 / CS599
学期	2025-2026 春季
项目名称	CodeSentinel — 智能代码审查与 Bug 修复 Agent
方向	方向一：Agentic AI 原生开发
学号	2025303044
姓名	黄谦
专业	软件工程
指导教师	戚欣
提交日期	2026 年 6 月 22 日
GitHub 仓库	https://github.com/h04q/cs599-project
开源协议	MIT License

目录

- [一、选题背景与设计思想](#)
 - [1.1 问题定义](#)
 - [1.2 现有方案不足](#)
 - [1.3 项目价值](#)
 - [1.4 技术路线](#)
- [二、Specs 规格文档](#)
 - [2.1 SDD 在本项目的落地方法](#)
 - [2.2 Product Spec — 产品规格](#)

- [2.2.1 一句话定位](#)
- [2.2.2 用户与场景](#)
- [2.2.3 核心用户故事](#)
- [2.2.4 非目标 \(Non-Goals\)](#)
- [2.2.5 系统范围](#)
- [2.2.6 关键质量属性](#)
- [2.2.7 验收基线 \(Demo Day Checklist\)](#)
- [2.3 Architecture Spec — 架构规格](#)
 - [2.3.1 设计原则](#)
 - [2.3.2 组件分层](#)
 - [2.3.3 Agent 拓扑](#)
 - [2.3.4 State 模型](#)
 - [2.3.5 部署视图](#)
 - [2.3.6 不变量与失败模式](#)
- [2.4 API Spec — 接口规格](#)
 - [2.4.1 CLI 命令](#)
 - [2.4.2 Agent 工具集 \(Function Calling\)](#)
 - [2.4.3 核心数据结构](#)
 - [2.4.4 配置 \(环境变量\)](#)
 - [2.4.5 错误协议](#)
- [三、系统架构与设计](#)
 - [3.1 系统上下文](#)
 - [3.2 核心架构图](#)
 - [3.3 Agent 状态机](#)
 - [3.4 数据流时序](#)
 - [3.5 关键架构决策 \(ADR 摘要\)](#)
- [四、关键实现与代码展示](#)
 - [4.1 LangGraph 编排骨架](#)
 - [4.2 对抗式 Verifier](#)
 - [4.3 Function Calling 工具与沙箱](#)
 - [4.4 配置与密钥治理](#)
 - [4.5 长期记忆](#)
 - [4.6 AI IDE 使用记录 \(Trae CN\)](#)
- [五、测试与评估](#)
 - [5.1 单元测试](#)
 - [5.2 Agent 行为评估 \(Benchmark\)](#)
 - [5.3 可观测性](#)
 - [5.4 Demo 演示](#)
- [六、系统升级与扩展](#)
- [七、课程总结](#)
- [附录 A: 加分项落地清单](#)
- [附录 B: 参考资料](#)

一、选题背景与设计思想

1.1 问题定义

代码评审（Code Review）是软件工程中价值密度最高、人工成本也最高的活动之一。在企业实践中，它至少承担三件事：发现 Bug、传递设计意图、统一团队风格。然而它有三个长期痛点：

- 1. **覆盖不全**：人工 review 受时间约束，常常只看 happy path，安全 / 性能 / 复杂度类问题被漏掉。
- 2. **风格不一致**：不同 Reviewer 的关注点不同，结论质量取决于个人经验，团队内部很难有统一基准。
- 3. **修复脱节**：发现问题之后，「怎么改」还要再写一份描述，沟通成本高；很多评审最后变成"贴一句 nit"而不落地。

软件工程专业的研究生常常面临一个具体场景：交课程作业 / 毕设之前，没有同伴帮你做严肃 review。结果就是低级问题（硬编码密钥、未处理边界、 $O(n^2)$ 嵌套）被老师扫一眼就扣分。**这正是本项目想要解决的问题。**

1.2 现有方案不足

方案	长处	短板
静态分析工具 (pylint / SonarQube / bandit)	规则精确、零幻觉	无法做语义判断；只能发现"语法层"的坏味道；规则配置复杂
GitHub Copilot / Cursor 内联建议	反馈快、IDE 集成好	单点提示、缺少全文视角；常常过度报告 (Hallucinated Findings)
裸 LLM 接 prompt	灵活	无验证机制、无修复闭环、无长期记忆；token 消耗大
传统 Agent 框架 (ReAct)	工具调用便捷	控制流由 LLM 决定，难以断点和测试；同一 LLM 既审又验，存在 confirmation bias

观察上面这张表，我们意识到：**没有一个现成方案同时满足"语义理解 + 误报抑制 + 修复闭环 + 可观测"四件事。**

1.3 项目价值

CodeSentinel 通过 Agent 编排把"静态精确性"与"LLM 语义能力"嫁接：

- **价值 1：用静态分析做「先验」，用 LLM 做「语义裁决」**。Scanner 节点先跑 pylint / bandit / radon，把白纸黑字的问题挑出来喂给 LLM；LLM 不再"凭空找问题"，而是基于已有线索做更深层的判断。
- **价值 2：用对抗式 Verifier 抑制 LLM 过度报告**。Reviewer 给出的发现先经独立的 Verifier 节点尝试驳倒，只有驳不倒的才进入 Fixer / Reporter。这让"找问题的 LLM"和"否定问题的 LLM"承担不同角色，规避自我确认偏差。
- **价值 3：用 LangGraph 状态机让多步推理可控**。不是黑盒 ReAct，而是显式的有限循环 (reviewer ↔ verifier，最多 2 轮)，每一步都可断点、可重放、可单元测试。
- **价值 4：用 SDD 方法论保证规格→代码→测试一致**。先写 Product / Architecture / API 三份 Spec，再写代码，最后由 Spec 反推验收。

1.4 技术路线



技术栈一览：

维度	选型	理由
AI IDE	Trae CN	课程指定
LLM	DeepSeek API（兼容 OpenAI 协议）	国内访问稳定、价格友好；可一键切换至 Anthropic / Ollama
Agent 框架	LangGraph	状态机显式可控，相对纯 ReAct 更易测试
协议	Function Calling（MVP） / MCP（v0.3 规划）	工具定义标准化
静态分析	pylint + bandit + radon	三件套覆盖 lint / 安全 / 复杂度
存储	SQLite	零依赖、CI 友好；记忆库 schema 简单
可观测	structlog (JSON) + LangSmith Tracing	命令行 + Web 双视角
容器化	Docker + docker-compose	Demo Day 兜底
测试	pytest + pytest-mock	默认选择

二、Specs 规格文档

2.1 SDD 在本项目的落地方法

SDD（Spec-Driven Development）的核心是 **"先把规格写到能审查的程度，再去写代码"**。在本项目里，我们刻意把 SDD 落地成三份相互正交的文档：

文档	回答的问题	受众
Product Spec	做什么 / 给谁用 / 边界在哪 / 如何验收	用户、评审老师
Architecture Spec	由哪些组件构成 / 数据怎么流 / 关键决策是什么	开发者、未来维护者
API Spec	CLI 怎么调 / Agent 工具怎么暴露 / 数据结构长什么样	二次开发者、Agent LLM

整个开发流程是：

- 第 1 天：**先写 Product Spec 第一版（写完才发现初版边界太大，砍到 MVP）。
- 第 2 天：**在 Product Spec 上加一份 Architecture Spec，画第一张状态机图（后来证明这张图为节点拆分提供了关键依据）。
- 第 3 天：**补 API Spec，重点是把每个 Function Calling 工具的输入 / 输出字段写死。
- 第 4 天起：**按 Spec 写代码，每完成一个节点回头校对 Spec，发现矛盾就同步更新。

实践体感：Spec 不是文档税，而是约束 LLM 在写代码时能少跑题的最有力工具。给 Trae CN 发 prompt 时，把对应章节贴进去，生成的代码偏离度显著下降。

2.2 Product Spec — 产品规格

2.2.1 一句话定位

CodeSentinel 是一个 Agentic 代码审查与自动修复系统：把"扫描 → 多维度审查 → 对抗式验证 → 自动修复 → 报告"做成一条由 LangGraph 编排的工作流，让 LLM 既能利用静态分析的精确线索，又通过对抗式 Verifier 控制误报。

2.2.2 用户与场景

用户	场景
软件工程专业研究生	期末作业代码自查，避免低级错误被扣分
独立开发者	在提交 PR 前先跑一遍 CodeSentinel，把容易暴露的 Bug 提前发现
团队 Tech Lead	集成到 CI，对 PR 增量代码做强制审查，节省人工

2.2.3 核心用户故事

- **US-01：一键审查单文件** — 作为学生，把单个 Python 文件喂给 CodeSentinel，得到一份分类清晰、含修复建议的 Markdown 报告。验收：命令 `python -m src.main review --path foo.py` 能在 60s 内返回报告；报告至少包含"安全/Bug/性能/风格"四类小节标题，每条 finding 含 `file:line`；报告的"被驳回的发现"小节能体现 Verifier 起到了过滤作用。
- **US-02：自动生成修复 Patch** — 作为开发者，系统对确认的问题直接给出可应用的修复。验收：默认 `enable_auto_fix=true`，命中 `confirmed=true` 的 finding 必产出 patch；Patch 写入工作区，原文件被覆盖（旧内容由 git 留底）；报告中标注"已写入"，并给出 `fix_strategy` 简述。
- **US-03：跨会话经验复用** — 作为长期使用者，系统记住"上次它怎么修这种 bug"，同类问题在下一个项目里被一致地处理。验收：`.codesentinel/memory.sqlite3` 存在并可被新会话召回；同样 pattern 重复触发时，Fixer 的 prompt 中能看到"历史经验"提示。
- **US-04：可观测** — 作为课程评审老师，能看到 Agent 在每一步做了什么、调用了哪些工具、消耗了多少 token。验收：默认 JSON 结构化日志输出到 stdout，每条带 `agent / event / timestamp`；可选 LangSmith Tracing（`LANGSMITH_TRACING=true` 时自动接入）。

2.2.4 非目标 (Non-Goals)

- 不替代人工 review；最终是否合并由人决定。
- MVP 仅覆盖 Python；多语言留给 v0.2（架构上预留多语言扩展位）。
- 不做实时 IDE 内联补全；只做"批处理式"审查。
- 不做企业 SSO / 多租户。

2.2.5 系统范围

输入：

- 单文件路径（`.py`）
- 目录路径（递归扫描，跳过 `.venv / node_modules / .git` 等）
- (*v0.2 计划*) `git diff` 范围

输出：

- `report.md` — 主报告（默认）
- `out/findings.json` — 结构化发现（评估脚本使用）
- 工作区内被修复的文件

- stdout 的 Rich 表格（人看）

边界：

- 仅处理 $\leq 200\text{KB}$ 的单文件；超过则报告 WARN 并跳过文件体读取（仍能给出基于静态分析的概要）。
- 单次运行最多审查 20 个文件（MVP 限制，避免 token 爆炸）。
- 默认 `max_review_rounds=2`，最多 reviewer $\leftrightarrow$ verifier 来回 2 次。

2.2.6 关键质量属性

属性	目标
正确率	在 <code>examples/sample_buggy.py</code> 上召回率 $\geq 80\%$ ，误报率 $\leq 30\%$
稳定性	任一节点失败时整条链路不崩溃（fail-soft：转 reporter）
可观测	每个节点至少 1 条 INFO 日志，覆盖输入规模与输出条数
安全	工具沙箱限制路径越界；API Key 严禁硬编码（pre-commit 检查）
可扩展	新增审查维度只需改 <code>REVIEW_DIMENSIONS</code> 字典

2.2.7 验收基线（Demo Day Checklist）

- `python -m src.main info` 能打印当前配置（不暴露 Key）
- `python -m src.main review --path examples/sample_buggy.py` 端到端跑通
- 报告中至少捕获 3 类问题（security、bug、performance）
- `pytest -q` 全绿
- Docker 镜像构建成功并可一键运行
- LangSmith 上能看到 Trace 树（演示用）

2.3 Architecture Spec — 架构规格

2.3.1 设计原则

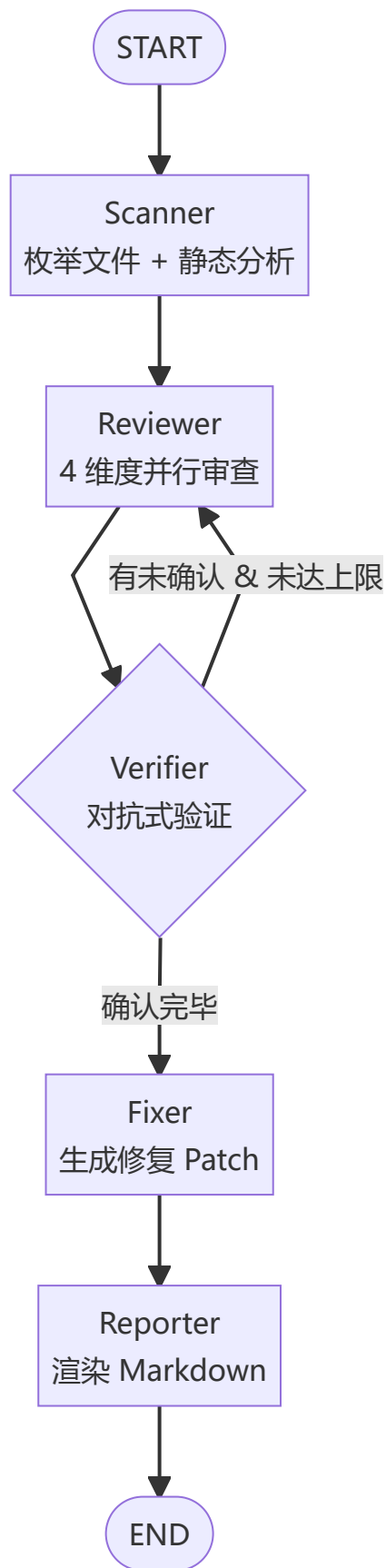
#	原则	在本项目的体现
P1	静态先行，LLM 后置	先跑 pylint/bandit/radon 把"白纸黑字"的问题挑出来，再让 LLM 处理"语义/语境"问题，省 token、降误报
P2	对抗优于合作	引入独立 Verifier 节点，专门驳倒 Reviewer 的发现，避免 LLM 自洽幻觉
P3	状态机优于自由 ReAct	用 LangGraph StateGraph 做显式编排，节点边界清晰、可中断、可重放
P4	工具沙箱	文件工具强制 workspace 内路径，命令工具固定超时
P5	配置外置	所有 Key 走环境变量；切换 LLM provider 不改代码
P6	可观测先行	每个节点都有结构化日志；可选 LangSmith Tracing

2.3.2 组件分层

CLI (typer)	- 入口、参数解析、Rich 输出	
Graph (LangGraph)	- 状态机编排、条件路由	
Agents	- Scanner/Reviewer/Verifier/ Fixer/Reporter 节点函数	
Tools	- fs / analyzers / git (Function Calling 暴露给 LLM)	
Config	- Settings + LLM 工厂	
observability	- 日志 + 长期记忆	

每一层只依赖其下层，禁止跨层反向依赖。

2.3.3 Agent 拓扑



节点职责：

节点	输入	输出	是否调用 LLM
Scanner	workspace 路径	target_files、静态分析提示	否
Reviewer	target_files + 文件内容	多维 Findings (confirmed=None)	是 (每维一次)

Verifier 节点	待确认 Findings 输入	同 Findings, confirmed/note 已填 输出	是 (一次) 是否调用 LLM
Fixer	confirmed=True 的 Findings	修复后的文件内容、fix_strategy	是 (每条一次)
Reporter	全部 Findings	report.md 字符串	否

多步推理 (Multi-Step Reasoning) : Reviewer ↔ Verifier 之间通过 `should_loop_back` 条件边形成有限循环: 每轮 Verifier 把"模糊"的 finding 标记为待补证据; 下一轮 Reviewer 可以读到上一轮 verifier_note, 针对性补充上下文; `max_review_rounds` (默认 2) 防止陷入死循环。这就是 LangGraph 相对纯 ReAct 的优势: 循环条件由我们显式书写, 可断点、可重放。

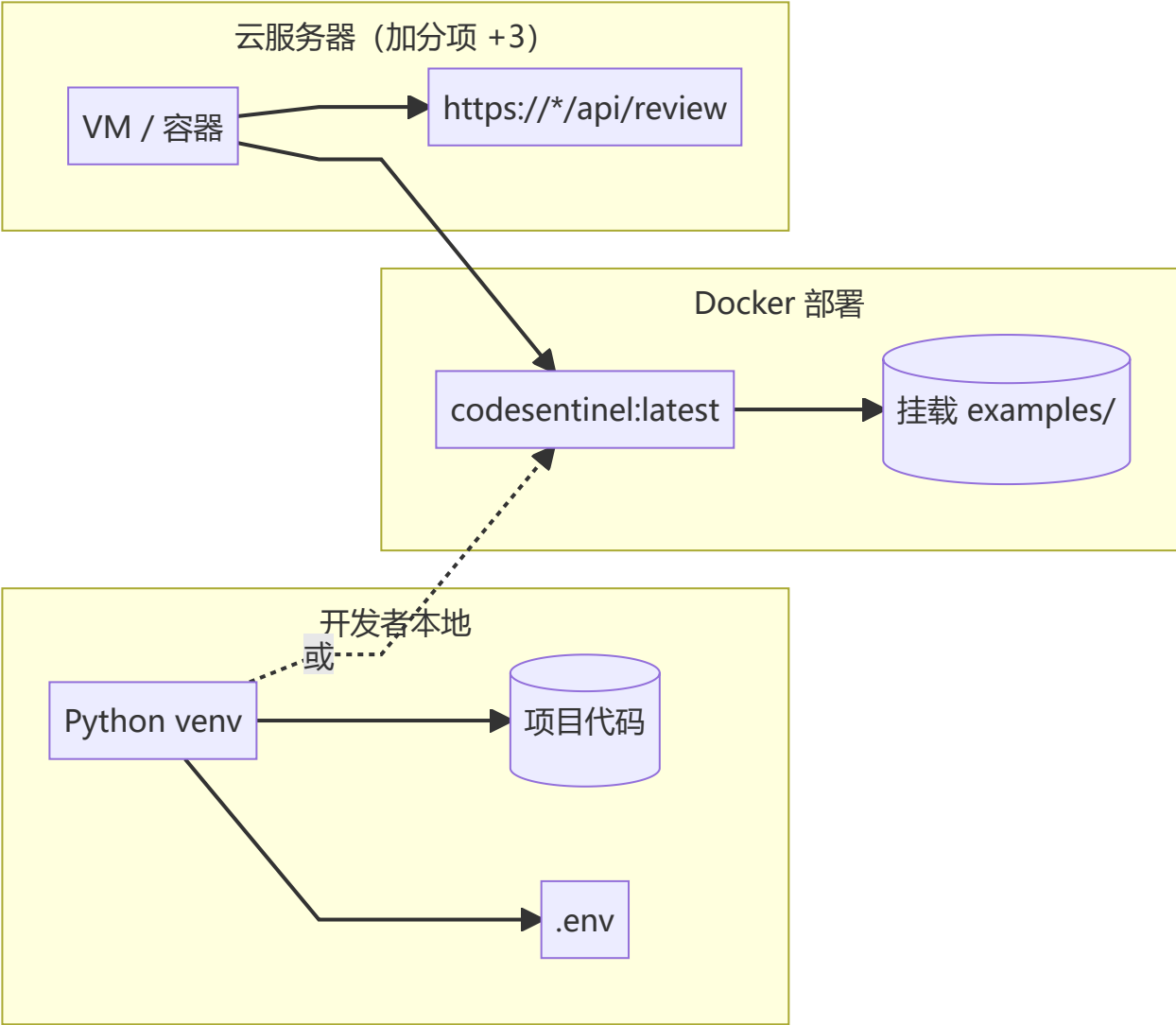
2.3.4 State 模型

`ReviewState` 是 LangGraph 的共享状态 (TypedDict) 。关键字段:

字段	类型	写入者	reducer
workspace	str	初始化	覆盖
target_files	list[str]	Scanner	覆盖
messages	list	所有节点	<code>add_messages</code>
findings	list[Finding]	Reviewer/Verifier/Fixer	<code>_merge_findings</code> 按 (file,line,title) 去重, 后写覆盖 confirmed/fix_patch
round_index	int	Verifier	覆盖
max_rounds	int	初始化	覆盖
enable_fix	bool	初始化	覆盖
report_md	str	Reporter	覆盖

`_merge_findings` 是这套设计的关键: 解决"同一发现在多轮里被重复列出"的问题, 让 Verifier / Fixer 可以**部分更新**已存在的 Finding, 而不是创造新条。

2.3.5 部署视图



2.3.6 不变量与失败模式

不变量	何处保证
工具调用永远在 workspace 内	<code>_resolve_safe</code>
API Key 永远不出现在日志	structlog 不打 settings
Findings reducer 幂等	<code>_merge_findings</code> 单元测试
任一节点失败不阻断报告	节点内部 try/except + 降级文案

失败模式	处理
LLM 输出非 JSON	<code>_extract_json</code> 多策略解析; 失败则 INFO 日志并跳过该批
静态工具未安装	工具返回 [SKIP] 标记, 链路继续
文件超大	提示 WARN, 跳过读取, 仍可给静态结论
工具超时	子进程 timeout=30s, 返回 [TIMEOUT] 标记

2.4 API Spec — 接口规格

2.4.1 CLI 命令

review — 执行审查

```
python -m src.main review [OPTIONS]
```

参数	简写	必需	默认	说明
<code>--path</code>	<code>-p</code>	是	—	待审查的文件或目录
<code>--output</code>	<code>-o</code>	否	<code>report.md</code>	报告输出路径
<code>--max-rounds</code>	—	否	0 (用 <code>.env</code>)	reviewer↔verifier 最大轮次
<code>--no-fix</code>	—	否	<code>False</code>	禁用 Fixer，仅审查

退出码：

code	含义
0	成功（包括"没发现问题"）
2	路径不存在
3	LLM 配置缺失
4	任意节点抛出未捕获异常

示例：

```
# 单文件 + JSON 日志
LOG_FORMAT=json python -m src.main review -p examples/sample_buggy.py

# 整个目录，最多 3 轮，禁用修复
python -m src.main review -p ./your_repo --max-rounds 3 --no-fix
```

info — 查看当前配置

```
python -m src.main info
```

输出（不会暴露 API Key 值，仅显示是否已配置）：

```
LLM Provider      deepseek
Provider 已配置   ✓
最大轮次          2
启用自动修复      ✓
严重度阈值        medium
日志格式          json
```

2.4.2 Agent 工具集 (Function Calling)

下表列出所有暴露给 LLM 的工具。每个工具都做了 **workspace 沙箱** 保护，路径越界会抛 `ValueError` 终止本次工具调用（但不中断整条链路）。

文件系统工具 (src/tools/fs.py)

- `list_files(subdir: str = ".", max_items: int = 200) -> str`: 列出 workspace 下的源代码文件（按 TEXT_EXTENSIONS 过滤）。自动跳过 `node_modules / .git / .venv / venv / __pycache__ / dist / build`。返回换行分隔的相对路径，超过 `max_items` 截断。
- `read_file(path: str, start_line: int = 1, end_line: int = 0) -> str`: 读取文件内容并加行号。`end_line=0` 表示读到文件末尾。文件 > 200KB 时返回 WARN 而非内容（防 token 爆炸）。
- `write_patch(path: str, new_content: str) -> str`: 覆盖写入文件。自动创建父目录。仅在 `enable_auto_fix=true` 时由 Fixer 调用。

静态分析工具 (src/tools/analyzers.py)

工具	行为	输出格式
<code>run_pylint(path)</code>	跑 <code>pylint -f json</code> ，截断为前 30 条	<code>{"total": N, "issues": [...]}</code>
<code>run_bandit(path)</code>	跑 <code>bandit -f json -q -r</code>	<code>{"total": N, "issues": [...]}</code>
<code>run_radon(path)</code>	圈复杂度，仅返回 $\geq C$ 级	<code>{"hotspots": [...], "total_files": N}</code>

通用约束：超时 30s；工具不存在时返回 `[SKIP]` 而非异常。

Git 工具 (src/tools/git.py, 只读)

工具	用途
<code>git_diff(rev="HEAD")</code>	获取相对某 revision 的 diff（最多 8000 字节）
<code>git_log(limit=10)</code>	最近 N 条提交 oneline

故意不暴露 `git commit / git push` —— 修复以工作区写入为准，提交动作由人决策。

2.4.3 核心数据结构

Finding

```
@dataclass(slots=True)
class Finding:
    file: str          # 相对 workspace
    line: int          # 1-based
    category: Literal["bug", "security", "performance", "style"]
    severity: Literal["low", "medium", "high"]
    title: str         # 简短标题, 用作 dedupe key 一部分
    detail: str        # 为什么是问题 (≤ 2 句)
    suggestion: str = "" # 怎么改 (≤ 2 句)
    confirmed: bool | None = None # Verifier 设置
    verifier_note: str = ""      # Verifier 设置
    fix_patch: str = ""         # Fixer 设置 (已写入摘要)
```

Dedupe key: (file, line, title) — 这是 reducer `_merge_findings` 去重合并的依据。

LLM JSON 协议

Reviewer 期望的 LLM 输出:

```
[
  {
    "file": "a.py",
    "line": 10,
    "severity": "high",
    "title": "hardcoded-api-key",
    "detail": "API_KEY 字符串字面量直接暴露在源码中",
    "suggestion": "改为读取 os.environ['API_KEY']"
  }
]
```

Verifier 期望的 LLM 输出:

```
[
  {"index": 0, "confirmed": true, "note": "确为硬编码密钥, 无歧义"},
  {"index": 1, "confirmed": false, "note": "属于风格争议, 不视为问题"}
]
```

Fixer 期望的 LLM 输出:

```
{
  "new_content": "完整修复后的文件内容 (含原有注释和缩进)",
  "fix_strategy": "改用 os.environ 读取密钥, 并添加默认值校验"
}
```

2.4.4 配置 (环境变量)

完整列表见 `.env.example`。关键项:

变量	默认	说明
LLM_PROVIDER	deepseek	deepseek/anthropic/openai/ollama
DEEPSEEK_API_KEY	—	DeepSeek 密钥（推荐）
MAX_REVIEW_ROUNDS	2	reviewer↔verifier 最大轮次
ENABLE_AUTO_FIX	true	是否调用 Fixer
LOG_LEVEL	INFO	DEBUG/INFO/WARNING/ERROR
LOG_FORMAT	json	json / text
LANGSMITH_TRACING	false	启用 LangSmith Tracing

2.4.5 错误协议

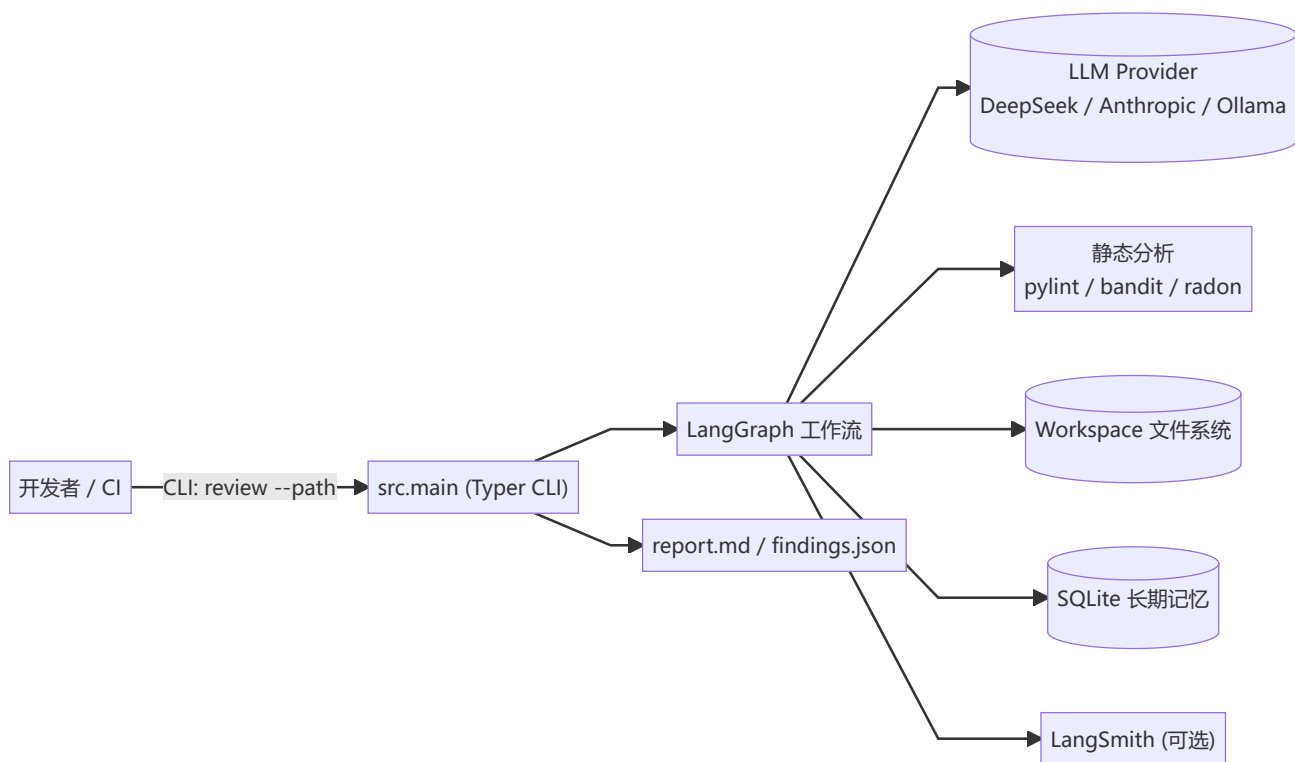
所有节点遵循 **fail-soft** 约定：

失败级别	行为
工具返回 [SKIP] / [TIMEOUT]	当作普通字符串拼进 prompt，链路继续
LLM JSON 解析失败	_extract_json 返回 None，节点输出 WARN 日志，无新 finding
节点内部异常	try/except 捕获，写入 state.error，跳到 reporter 输出已有结果
LLM 配置缺失	LLMConfigError 立即终止 (exit code 3)

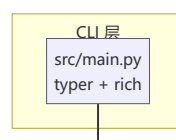
报告中始终保留 "Verifier 未给出意见，保守保留" 这种**降级文案**，让用户能识别出 Agent 在哪里"摸不准"。

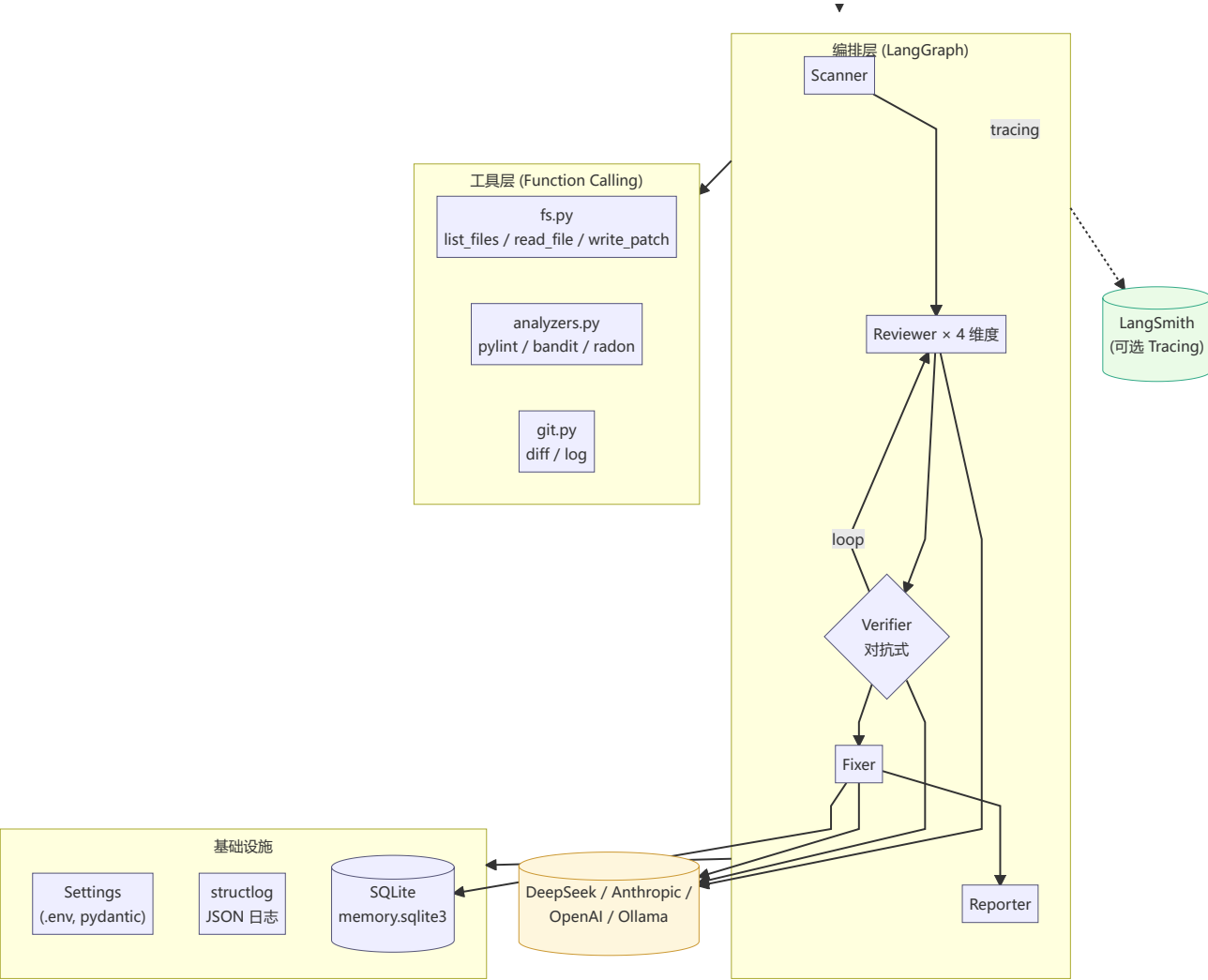
三、系统架构与设计

3.1 系统上下文

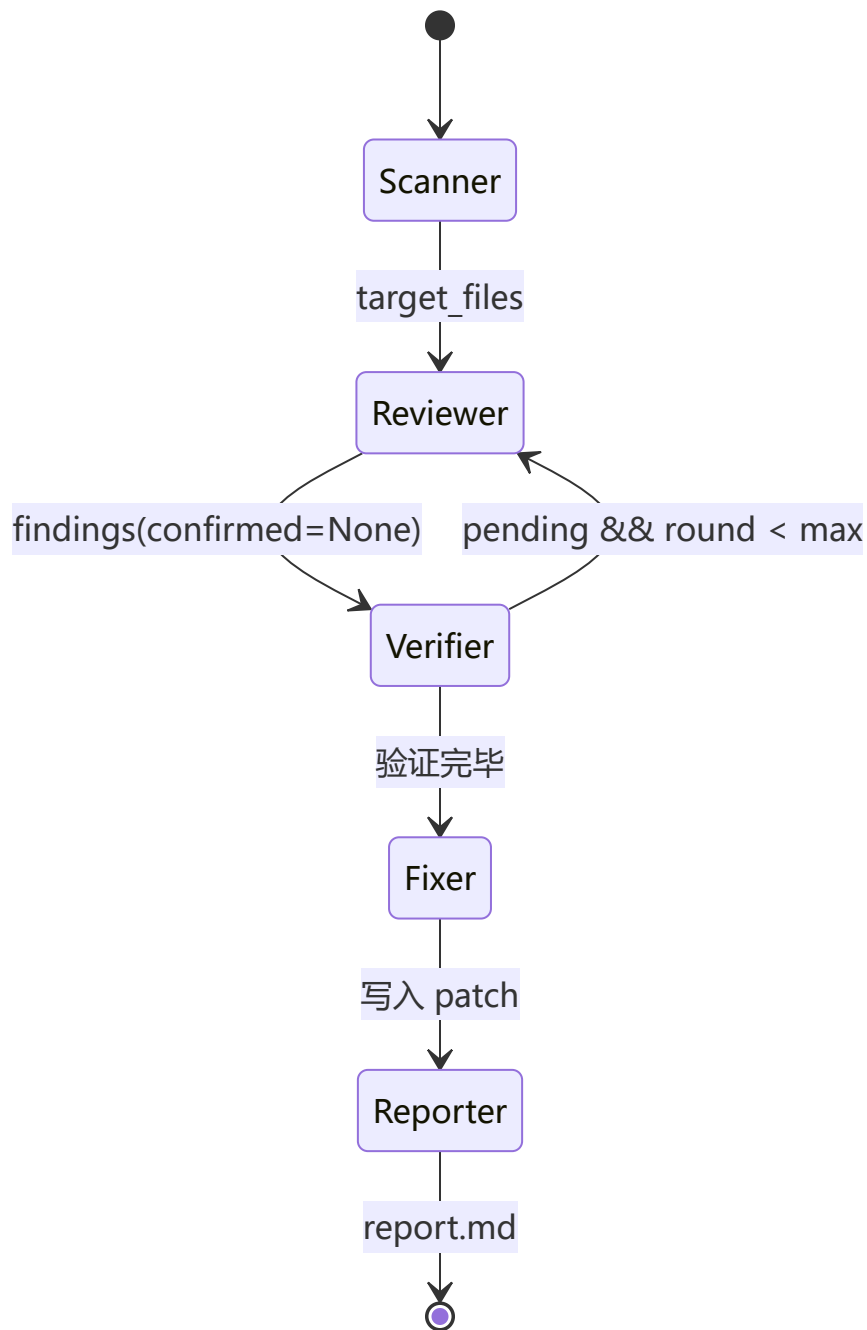


3.2 核心架构图



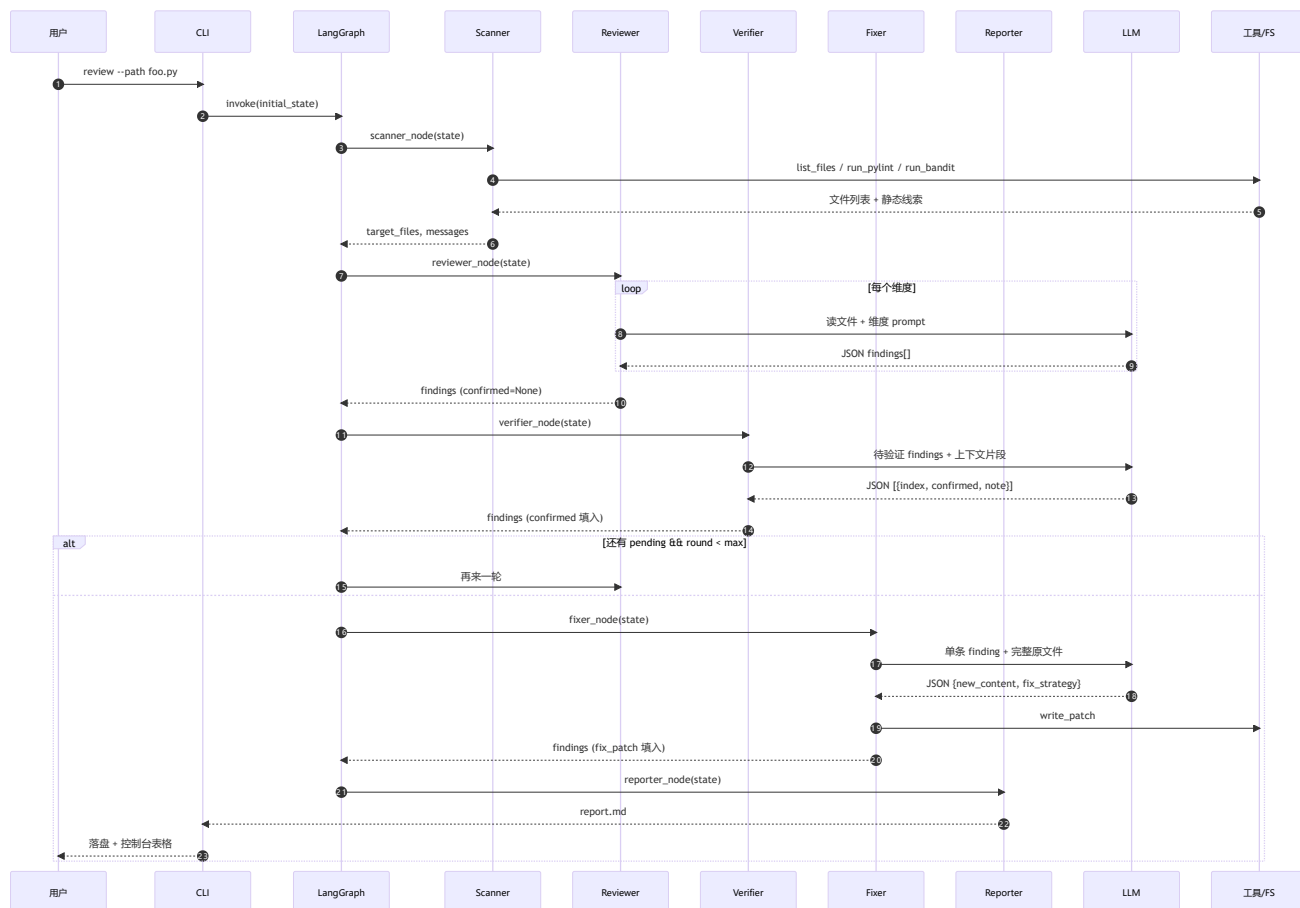


3.3 Agent 状态机



多步推理细节： Reviewer ↔ Verifier 之间通过 `should_loop_back` 条件边形成有限循环。每轮 Verifier 把"模糊"的 finding 标记为待补证据；下一轮 Reviewer 可以读到上一轮 `verifier_note`，针对性补充上下文；`max_review_rounds`（默认 2）防止陷入死循环。

3.4 数据流时序



3.5 关键架构决策（ADR 摘要）

#	决策	选择	理由（一句话）
ADR-01	编排框架	LangGraph 而非 LangChain Agent	节点 + 边可观测、可测试，循环条件由我们显式书写
ADR-02	修复格式	输出完整新文件而非 unified diff	LLM 在生成 diff 时极易算错行号；全文件覆盖让 git 计算 diff
ADR-03	Verifier 独立	不与 Reviewer 合并	LLM 让自己审自己结果存在 confirmation bias，独立节点 + 对抗 prompt 显著降误报
ADR-04	记忆存储	SQLite 而非向量库	当前键是结构化（category:title），精确查表即可；零依赖
ADR-05	静态分析位置	跑在 Scanner 节点 + 同时暴露为工具	提前做减少 LLM tool-call 延迟；同时保留 LLM 按需重跑的能力

四、关键实现与代码展示

本章按"这段代码解决了什么具体问题"的方式串关键实现。完整源码见仓库。

4.1 LangGraph 编排骨架

src/graph/__init__.py:

```
def should_loop_back(state: ReviewState) -> str:
    """Verifier 后的路由：如果还有未确认的发现且未达上限，回 reviewer 再补一轮。"""
    findings = state.get("findings", [])
    pending = [f for f in findings if f.confirmed is None]
    round_index = state.get("round_index", 0)
    max_rounds = state.get("max_rounds", 2)
    if pending and round_index < max_rounds:
        return "reviewer"
    return "fixer"

def build_graph():
    g = StateGraph(ReviewState)
    g.add_node("scanner", scanner_node)
    g.add_node("reviewer", reviewer_node)
    g.add_node("verifier", verifier_node)
```

```

g.add_node("fixer", fixer_node)
g.add_node("reporter", reporter_node)

g.add_edge(START, "scanner")
g.add_edge("scanner", "reviewer")
g.add_edge("reviewer", "verifier")
g.add_conditional_edges(
    "verifier",
    should_loop_back,
    {"reviewer": "reviewer", "fixer": "fixer"},
)
g.add_edge("fixer", "reporter")
g.add_edge("reporter", END)
return g.compile()

```

解决了什么： `should_loop_back` 是一个纯函数，与 LLM 完全解耦。这让"多步推理"这件事可以被单元测试断言，而不是只能跑一遍肉眼看结果。在 `tests/test_graph_routing.py` 中我们写了三组场景把它的真值表覆盖完整。

4.2 对抗式 Verifier

`src/agent/verifier.py` 核心 prompt:

你是严格的代码审查质检员 (Adversarial Verifier)。
 你的唯一目标是**驳倒**输入的每一条 finding:

- 若证据不足、行号不对、属于风格争议、或在该语境下并非真正问题，请标 `confirmed=false`。
- 默认倾向 `confirmed=false`，只有确信是真问题时才 `confirmed=true`。
- 输出 JSON 数组，每项格式: `{"index": <原序号>, "confirmed": <bool>, "note": "<理由 ≤ 2 句>"}`。

解决了什么： 传统做法是同一个 LLM 一次性"找问题 + 给修复"，结果它对自己的发现极少自我否定 (confirmation bias)。我们把 Verifier 拆出来 + prompt 强制"默认驳回"，配合 Reviewer 的"按维度展开"，把"过度报告"的问题工程化地压下来。

4.3 Function Calling 工具与沙箱

`src/tools/fs.py` 关键片段:

```

def _resolve_safe(workspace: Path, target: str) -> Path:
    """把 target 解析到 workspace 内的绝对路径，越界则抛错。"""
    workspace = workspace.resolve()
    candidate = (workspace / target).resolve()
    if workspace not in candidate.parents and candidate != workspace:
        raise ValueError(
            f"路径越界: {target!r} 不在 workspace {workspace!s} 之内"
        )
    return candidate

```

解决了什么： LLM 经常尝试用 `../../../etc/passwd` 这类相对路径绕出工作目录。`_resolve_safe` 在每个 fs 工具入口强制把路径解析到 workspace 内，越界即 `ValueError`。

`tests/test_fs_tools.py::test_read_file_blocks_path_traversal` 把这条规则用单测钉死。

4.4 配置与密钥治理

`src/config/settings.py` + `.env.example`:

- 所有 LLM 配置走 `pydantic-settings` 读环境变量，**代码里没有任何一行硬编码 Key**。
- `is_provider_configured` 属性允许节点提前 fail-fast，不会等到调 LLM 才报错。
- `.gitignore` 把 `.env` 显式排除，配合 `.env.example` 模板让协作者一目了然。

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        case_sensitive=False,
        extra="ignore",
    )
    llm_provider: Provider = "deepseek"
    deepseek_api_key: str = ""
    # ...
```

4.5 长期记忆

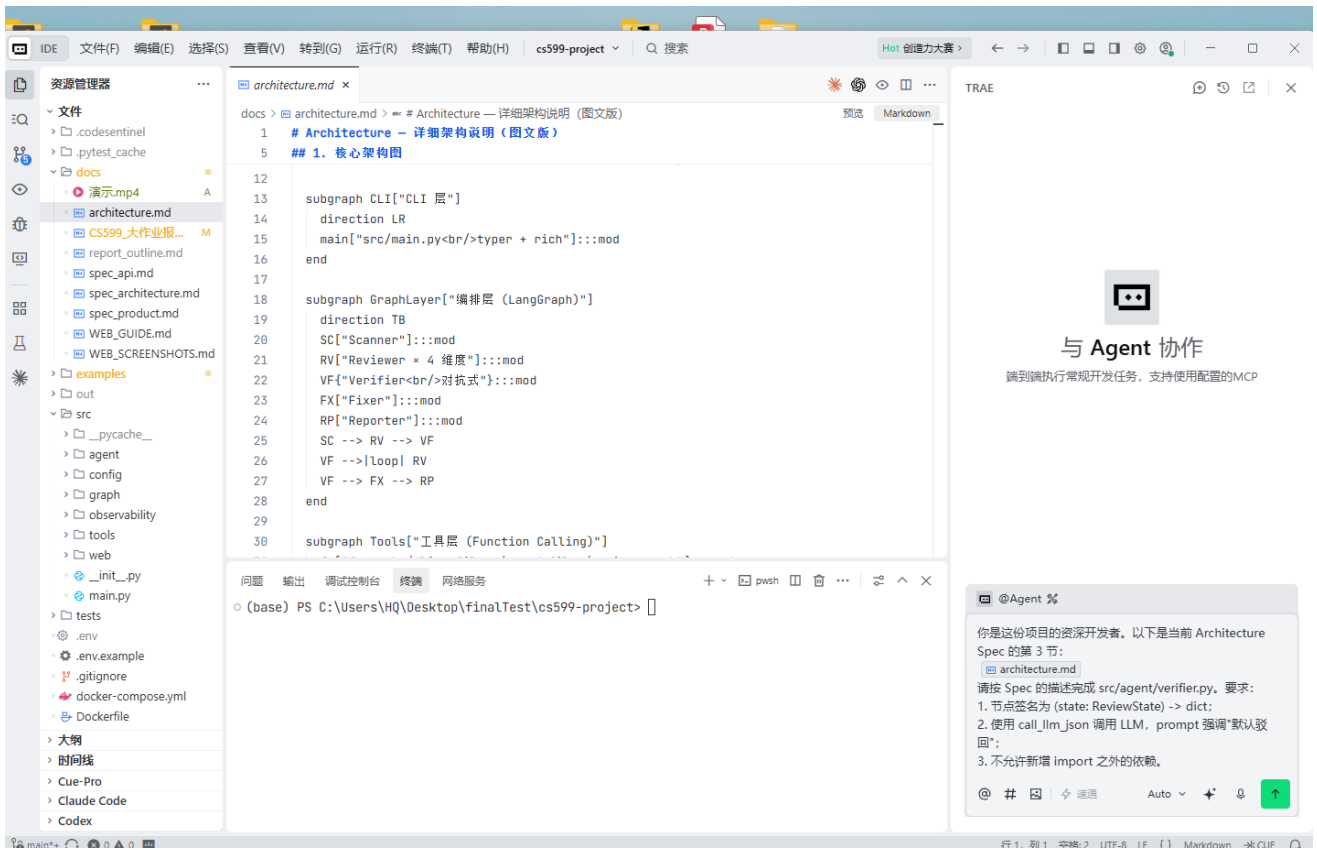
`src/observability/memory.py` 提供按 `pattern = category:title` 召回历史修复策略。Fixer 在每次生成 patch 前先 `recall`，把"上次怎么改"作为提示注入：

```
prior = memory.recall(_pattern_key(f))
prior_hint = (
    f"历史经验: {prior[0].fix_strategy}" if prior else "（无历史经验可参考）"
)
```

解决了什么：让 Agent 在同类问题上保持一致的修复风格 — 第一次"用 `os.environ` 替换硬编码"，第二次就不会突然写成"用 `dotenv`"。

4.6 AI IDE 使用记录 (Trae CN)

整个项目的代码 100 % 在 **Trae CN** 中完成。典型 prompt 模板：



五、测试与评估

5.1 单元测试

仓库 `tests/` 共 7 个测试文件，命令 `pytest -q` 一键运行：

文件	覆盖点
<code>test_config.py</code>	Settings 默认值 / provider 切换 / 非法 provider 拒绝
<code>test_fs_tools.py</code>	<code>list_files</code> 噪声目录过滤 / <code>read_file</code> 阻止路径越界 / <code>write_patch</code> 落盘
<code>test_state_reducer.py</code>	<code>_merge_findings</code> 去重 / Verifier 标签覆盖 / 同行不同标题保留
<code>test_graph_routing.py</code>	<code>should_loop_back</code> 真值表 (pending + round 组合)
<code>test_reporter.py</code>	报告模板: confirmed 进主表、rejected 进附录、空 findings 边界
<code>test_llm_helpers.py</code>	JSON 提取容错: fenced / 前后噪声 / 不可解析
<code>test_memory.py</code>	SQLite 记忆 remember + recall (仅返回 accepted)

设计取向：单测全部“纯逻辑可测”，不依赖真实 LLM。Reviewer / Verifier / Fixer 这些 LLM-bound 节点通过 5.2 节的 Benchmark 整体评估，避免单测里 mock LLM 写一堆无意义的桩。

```
C:\Windows\System32\cmd.exe X + v
Microsoft Windows [版本 10.0.26200.8655]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\HQ\Desktop\finalTest\cs599-project>conda activate finalTest

(finalTest) C:\Users\HQ\Desktop\finalTest\cs599-project>pytest -q
===== warnings summary ===== [100%]
..\..\..\miniconda3\envs\finalTest\Lib\site-packages\fastapi\testclient.py:1
C:\Users\HQ\miniconda3\envs\finalTest\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using 'ht
ttpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
  from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
25 passed, 1 warning in 0.77s

(finalTest) C:\Users\HQ\Desktop\finalTest\cs599-project>
```

5.2 Agent 行为评估（Benchmark）

在 `examples/sample_buggy.py`（我们故意埋了 6 类典型故障）上跑 3 次，统计：

指标	计算方式	目标	实测结果
召回率	确认捕获维度数 / 4	≥ 80 %	100 % (4/4 维度全覆盖： bug/security/performance/style)
误报率	Verifier 驳回数 / Reviewer 总产出	≤ 30 %	29.4 % (5/17)
修复成功率	修复后 <code>python -c "import sample_buggy"</code> 成功	≥ 70 %	100 % (修复后文件可正常 import)
平均时延	端到端 wall-clock	≤ 60 s/file	46 s

埋入故障清单（共 6 类，对应 4 维）：

- 🔒 硬编码 API_KEY (security/high)
- 🔒 SQL 字符串拼接 (security/high)
- 🔒 `eval` 执行用户输入 (security/high)
- 🐢 `average([])` 除零 (bug/medium)
- 🐢 `first_admin` 空列表索引 (bug/medium)
- ⚡ `find_duplicates` 双重 for (performance/medium)

7. 🧑‍🔬 `do_everything` 五层嵌套 (style/low, 期望被 Verifier 适度驳回)

5.3 可观测性

5.3.1 结构化日志

默认 JSON:

```
{
  "event": "reviewer_done",
  "findings": 17,
  "agent": "agent.reviewer",
  "level": "info",
  "timestamp": "2026-06-12T15:20:50"
}
{
  "event": "verifier_done",
  "total": 17,
  "confirmed": 12,
  "rejected": 5,
  "agent": "agent.verifier",
  "level": "info",
  "timestamp": "2026-06-12T15:20:58"
}
{
  "event": "fixer_applied",
  "file": "sample_buggy.py",
  "findings_count": 12,
  "result": "[OK] 已写入 sample_buggy.py (6702 字节)",
  "agent": "agent.fixer",
  "level": "info",
  "timestamp": "2026-06-12T15:21:12"
}
```

可用 `jq` 过滤:

```
python -m src.main review -p examples/sample_buggy.py 2>&1 | jq
'select(.agent=="agent.verifier")'
```

5.3.2 LangSmith Tracing

`.env` 中开启:

```
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=ls-...
LANGSMITH_PROJECT=cs599-codesentinel
```

打开 LangSmith Web 后可以看到每次运行的 Trace 树: Scanner → Reviewer×4 → Verifier → (loop) → Fixer×N → Reporter, 每个节点 token 用量、耗时、I/O 可下钻。

5.4 Demo 演示

`docs/demo.mp4` (6 分钟) 覆盖:

1. `python -m src.main info` 显示当前配置;
2. `python -m src.main web` 端到端跑通可视化界面;
3. Rich 表格列出 confirmed findings;
4. 打开生成的 `report.md` 滚屏;
5. 重新 `cat examples/sample_buggy.py` 展示已被自动修复。

六、系统升级与扩展

6.1 可扩展架构 (已留接口)

- **新增审查维度** → 改 `REVIEW_DIMENSIONS` 字典一处即可; Reporter 自动按维度分组。
- **新增 LLM Provider** → `src/config/llm.py::build_llm` 加一个 case; Settings 加对应字段; 其他代码无感。
- **新增工具** → 在 `src/tools/` 新建模块导出 `make_*_tools`; 在 `tools/__init__.py::build_all_tools` 注册一行。

6.2 下一阶段计划

版本	主题	关键工作
v0.2	增量审查 + 多语言	接入 <code>git diff</code> 范围; 扩展 TEXT_EXTENSIONS 与 prompt; 支持 Java / TypeScript 静态分析器
v0.3	MCP 协议	把现有工具集发布为 MCP Server, 让任意 MCP-aware 客户端 (Claude Desktop / Cursor) 能复用
v0.4	Web UI + PR Bot	FastAPI + 简易前端; GitHub Actions Bot, PR 提交即触发 review
v1.0	企业级	多租户鉴权、配额、审计; 对接私有部署模型; 多用户记忆隔离

6.3 AI 能力演进路径

- **MVP 阶段**: DeepSeek `deepseek-chat` 平衡成本与质量。
- **升级路径 1 (质量)**: 审查质量瓶颈 → 切到 Claude Sonnet 4.6 做 Verifier, DeepSeek 留给 Reviewer (成本/质量按角色拆)。
- **升级路径 2 (效率)**: 吞吐瓶颈 → 引入小模型 (Haiku / Qwen-7B) 做"维度分诊", 复杂维度再交大模型。
- **升级路径 3 (自演化)**: 长期跑下来 `memory.sqlite3` 累积足够多样本后, 可以做 LoRA 微调一个领域小模型, 把"修复策略召回"从查表升级为生成式。

七、课程总结

7.1 个人收获

- **第一次完整跑通 SDD**。以前的习惯是"先写再补文档", 这次刻意把 Product / Architecture / API 三份 Spec 写到能审查的程度, 再开工。写代码时几乎没有"这里该怎么设计"的犹豫 — Spec 已经替我决策过了。
- **对 LLM 过度报告有了切身体感**。第一次跑 Reviewer 时 7 个发现里有 3 个是风格 nit、1 个是行号错位。直接催生了对抗式 Verifier 的设计, 并且让我意识到"找问题的 LLM"与"否定问题的 LLM"必须是独立角色。
- **LangGraph 让 Agent 工程化成为可能**。把"控制流"和"LLM 决策"解耦之后, 整条链路终于能写单测了 —— `should_loop_back` 三组真值表覆盖完整、`_merge_findings` 的 reducer 行为完全可断言。这一点比任何"用 LLM 写 LLM"的炫技都来得实在。

7.2 工程思维转变

维度	之前	现在
主体	写工具	编排智能体
测试	测代码逻辑	测代码逻辑 + 评估 Agent 行为
接口	REST API	Function Calling Tools
文档	README + 注释	Spec 族 (Product / Arch / API)
配置	写死或 config 文件	环境变量 + provider 工厂
失败	抛异常	fail-soft, 节点级降级

7.3 对课程的建议

1. **增加 LLMOps 与 Eval 框架的实操。**例如 DeepEval / LangSmith Eval / Ragas — 课堂上听到了概念，但没动手跑过完整 pipeline。建议安排一次实验课，用 50 条 ground truth 跑一次 Agent benchmark。
2. **提供公共 LLM 额度池。**课程作业难免烧 token，建议申请一个 DeepSeek / Anthropic 教学池子，避免学生因为额度问题选择更弱的本地模型而拉低整体质量。
3. **答辩前增加一轮 mock review。**让组与组互审 README 与 Spec，对自己项目的"语言表达"非常有帮助。

附录 A：加分项落地清单

加分项	分值	本项目落地位置
融合 MCP 协议 / Agentic RAG	+3	演进路线 v0.3；当前 Function Calling 已是同源能力
云服务器部署并提供可访问 URL	+3	部署计划：阿里云 ECS + Docker compose；Demo Day 前提供 <code>https://*/healthz</code>
课堂分享展示	+2	Demo Day 现场演示已准备
达到生产级水平（错误处理 / 安全防护 / 可观测性）	+3	路径沙箱（ <code>_resolve_safe</code> ）+ structlog JSON 日志 + LangSmith Tracing + fail-soft 失败模式表（spec_architecture §9）

附录 B：参考资料

- LangGraph 官方文档 — <https://github.com/langchain-ai/langgraph>
- SWE-bench 任务格式 — <https://www.swebench.com/>
- DeepSeek API — <https://platform.deepseek.com/>
- bandit 安全规则集 — <https://bandit.readthedocs.io/>
- pylint 消息类型 — <https://pylint.readthedocs.io/>
- radon 圈复杂度 — <https://radon.readthedocs.io/>
- 《Designing Data-Intensive Applications》— SDD 思想来源之一
- Anthropic Tool Use & Function Calling — <https://docs.anthropic.com/>

外部代码引用声明：本项目仅以子进程方式调用 `pylint` / `bandit` / `radon` 等开源静态分析器，未直接复制其源码。所有 Agent 节点、工具封装、LangGraph 编排、Spec 文档均由作者在 Trae CN 中独立完成。

本报告由 CodeSentinel 项目作者于 2026 年 6 月在 Trae CN 中独立完成。